

SENTINEL

Technical Workflow & Integration Diagram Specification

Gujarat CCTV Integration Hackathon 2026 | Category 1 (Academic / Research / Startup)

Team Name: KodeMatrix

Team Members: Devam Shah, Krishit Shah, Moksheet Shah, Abhishek Shah, Yash Rathod

Document: Workflow & Integration Diagram Specification

Project: Sentinel: Statewide CCTV Integration & ANPR Surveillance Engine

Repository: [DevamShah1211/Sentinel-Hackathone](#)

ASCII Diagram Legend: [BLOCK] = system component or stage | | = vertical flow | ▼ = direction of data flow | └─ = sub-item branch | └─ = last sub-item in a branch | ➡ = directed connection | └─┬─ = split / decision point

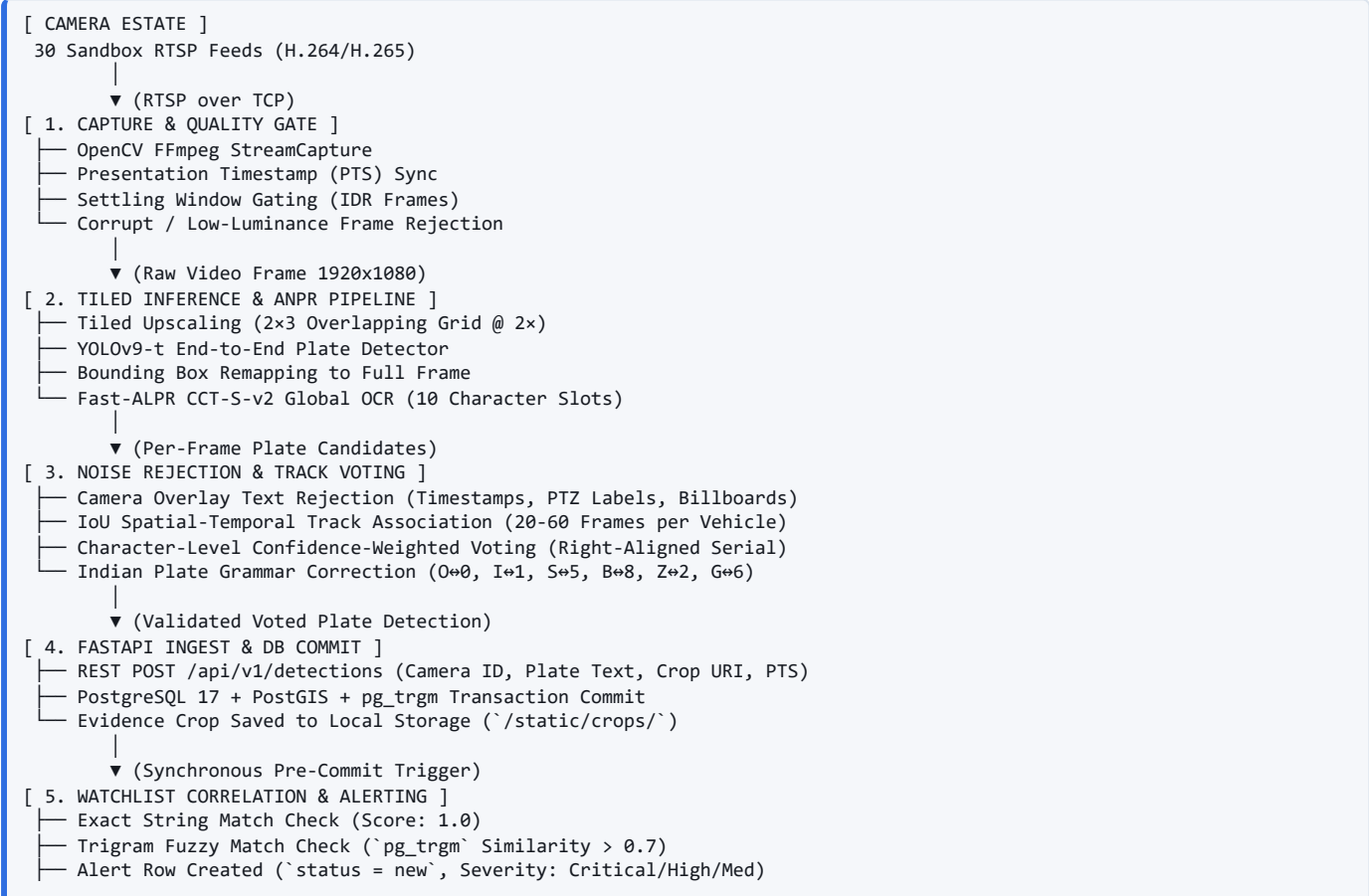
1. Executive Summary

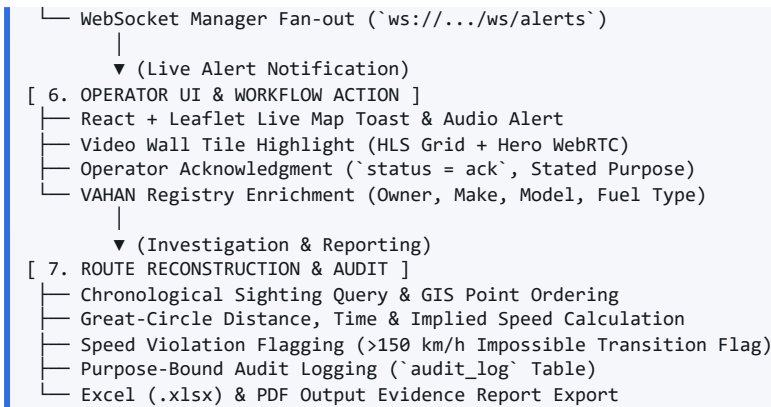
Sentinel is designed as a modular, edge-first statewide CCTV integration platform that ingests camera streams from heterogeneous departmental network feeds, performs continuous local license plate recognition (ANPR), correlates detections against real-time enforcement watchlists, dispatches low-latency WebSocket notifications, and reconstructs vehicle travel corridors on GIS maps with comprehensive audit logging.

This document details the operational workflow, end-to-end data pipelines, microservice integration points, database schemas, and edge-to-cloud scalability topology.

2. End-to-End Operational Workflow Diagram

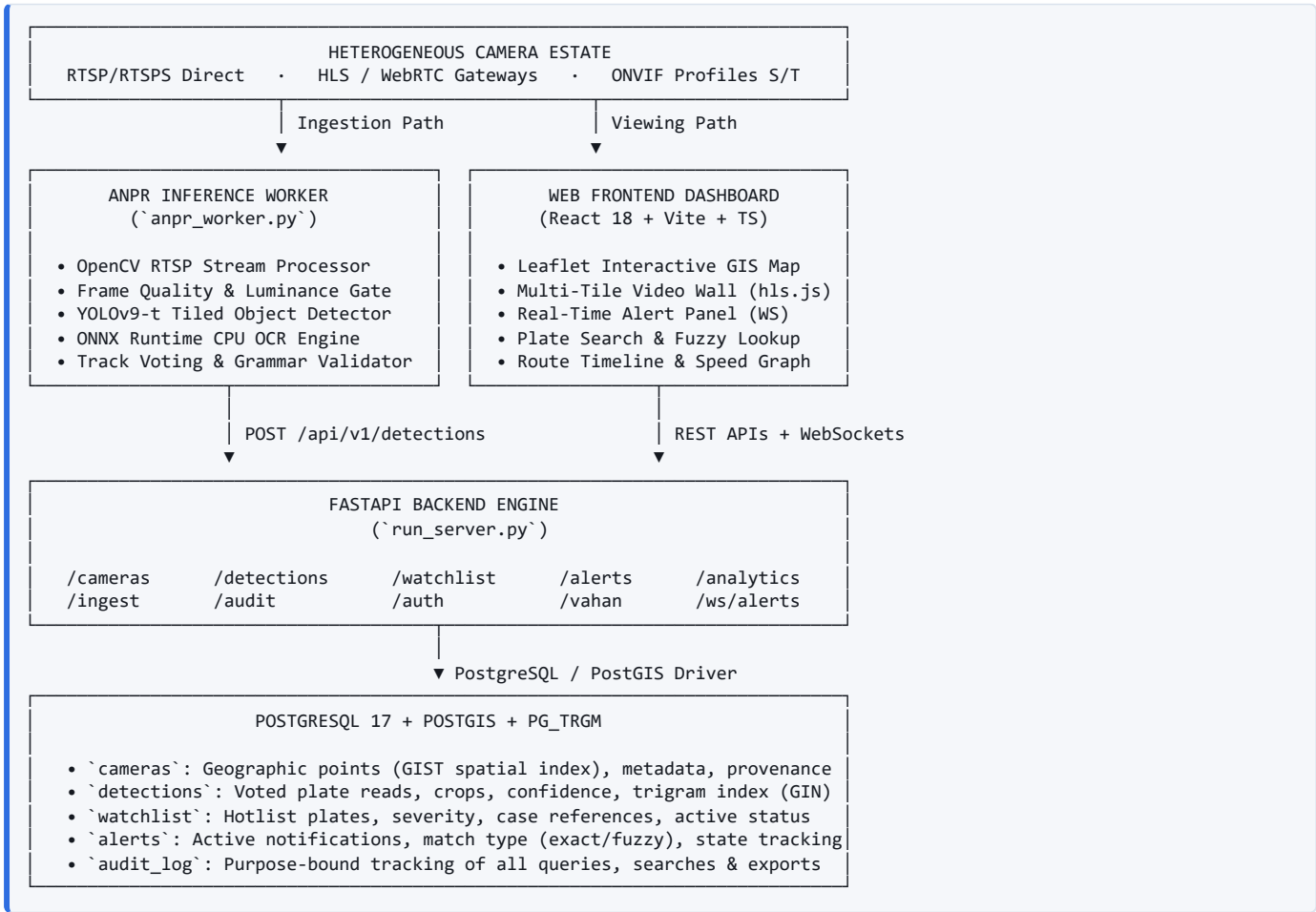
The sequence below illustrates how a live RTSP feed is ingested, processed, matched against enforcement watchlists, and surfaced to operators:





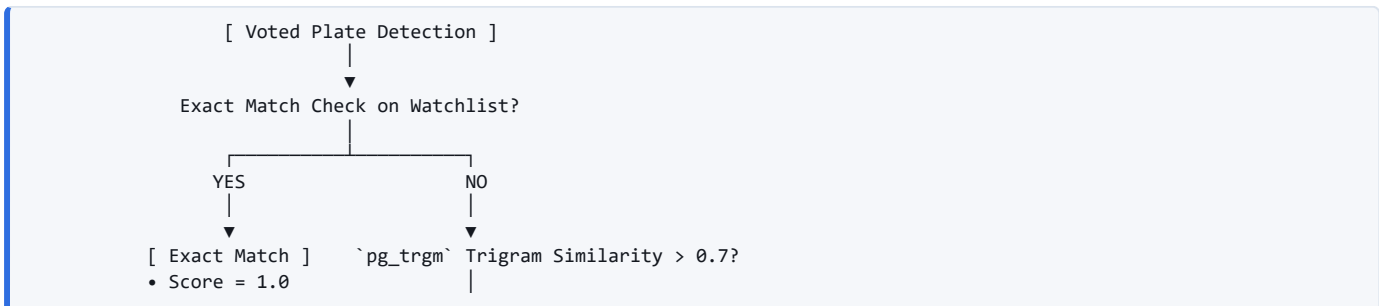
3. Microservice & System Integration Topology

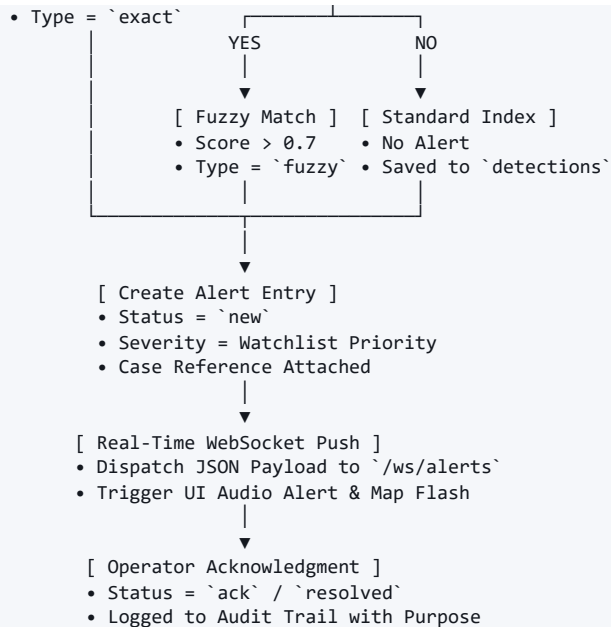
Sentinel separates video ingestion and compute-heavy inference from the API web backend and frontend UI layer:



4. Watchlist Correlation & Alert Workflow

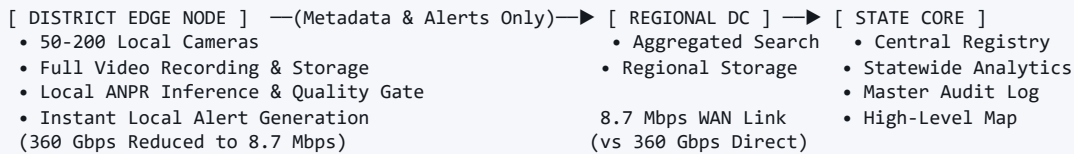
The diagram below details the exact decision tree executed whenever a number plate is indexed:





5. Statewide Edge-First Deployment Topology (Scale to 80,000 Cameras)

To prevent centralized network backhaul saturation (192 Gbps raw video bottleneck), Sentinel utilizes an **edge-first architecture**:



6. Integration Contract Interfaces

6.1 VAHAN Vehicle Registry Adapter Interface

- **Endpoint:** `GET /api/v1/vahan/vehicle/{plate_number}`
- **Mock / Integration Adapter:** Contract-first architecture returning registration date, owner name, vehicle class, fuel type, engine number, and insurance validity.

6.2 Audit Trail Verification Interface

- **Endpoint:** `GET /api/v1/analytics/audit`
- **Enforcement:** Logs user ID, role, client IP, action type (`SEARCH` , `ROUTE_RECONSTRUCT` , `EXPORT`), timestamp, and explicit **stated legal purpose**.